SE446: Big Data Engineering

Week 9A: Spark MLlib — Machine Learning at Scale

Prof. Anis Koubaa

SE 446
Alfaisal University
https://github.com/aniskoubaa/big_data_course

Spring 2026

Instructor Notes

Welcome everyone to Week 9A. Today we are talking about machine learning at scale using Spark MLlib. Now, I know many of you have already taken an ML course, and that is exactly the point — you already know the algorithms. What we are going to learn today is how to take that same knowledge and apply it when your data is too big for one machine. Think of it this way: you already know how to cook, and today I am teaching you how to cook for a thousand people. The recipes are the same, but the kitchen is very different. We will use the Chicago Crimes dataset you have been working with all semester, so the data should feel familiar. By the end of today, you will be able to build a full ML pipeline in Spark. Let us get started.

Today's Agenda

Instructor Notes

Here is the roadmap for today. We have five major sections. First, we will talk about why you even need distributed ML — what breaks when you try scikit-learn on big data. Second, we will cover the ML Pipeline API, which is the backbone of how Spark organizes ML workflows. Third, feature engineering — how to prepare your data for ML in Spark. Fourth, model training with three different classifiers. And fifth, evaluation, model comparison, and hyperparameter tuning. Notice how this mirrors what you would do in any ML project: prepare data, train models, evaluate. The only difference is that everything runs on a cluster instead of your laptop.

Outline

You Already Know Machine Learning

*This is **not** your first ML course. Let's build on what you know.*

What You Already Know

- Train/test split
- Feature engineering
- Logistic Regression, Random Forest
- Accuracy, Precision, Recall, F1
- Cross-validation, GridSearch
- scikit-learn API

What's New Today

- **Why** scikit-learn breaks at scale
- **How** distributed training works
- The Spark MLlib Pipeline API
- Same algorithms, different execution
- Training on terabytes, not gigabytes
- When to use which tool

Today's Guiding Question

Same math, same intuition — but what changes when your data doesn't fit on one machine?

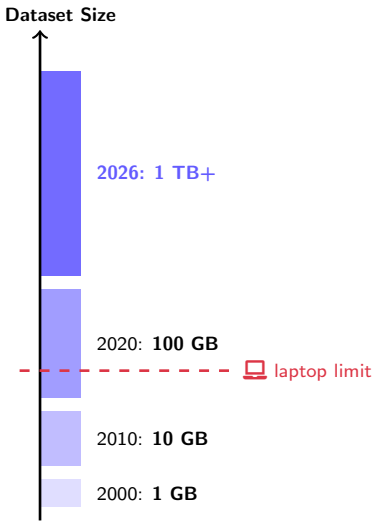
Instructor Notes

I want to be very clear about something right from the start. Look at the left column — train/test split, feature engineering, Random Forest, evaluation metrics. You already know all of this. This is not a new ML course. Now look at the right column. What is new is the **why** and the **how** of doing ML when your data does not fit on a single machine. The guiding question at the bottom is the key: same math, same intuition, but what changes? The answer, as we will see, is the execution model. The algorithms are identical. The way data flows through the system is what changes. So relax — if you understood scikit-learn, you will understand MLlib. We are just adding a new execution layer on top of concepts you already have.

ML Is Everywhere — But Data Is Getting Bigger

Real problems that use ML on large data:

- **Crime prediction**: Can we predict whether a crime will lead to arrest? *(Today's task)*
- **Healthcare**: Detect disease from millions of patient records
- **Finance**: Fraud detection on billions of transactions per day
- **Web**: Recommendation systems for 100M+ users



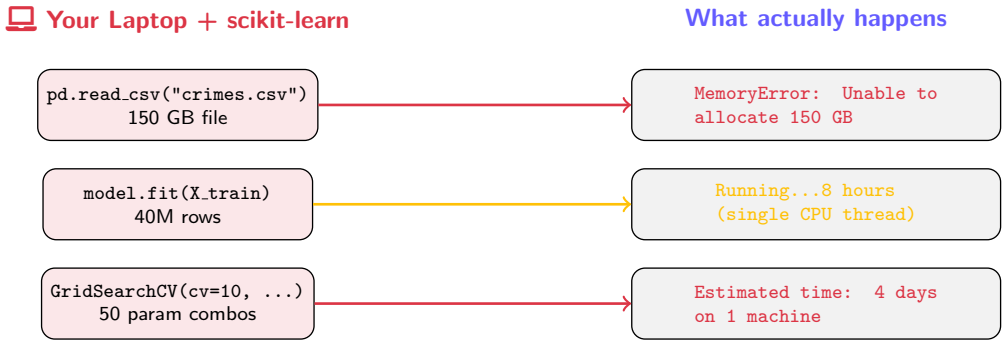
The Challenge

These datasets do **not fit on a laptop**.
A single machine crashes, stalls, or takes days.

Instructor Notes

Look at the bar chart on the right side of this slide. See the dashed red line labeled laptop limit? That is roughly where your machine maxes out — somewhere around 10 to maybe 50 gigabytes if you have a beefy laptop. Now look at where data sizes are in 2026 — we are well past one terabyte for many real-world datasets. The examples on the left are not hypothetical. Netflix, your bank, hospitals — they all run ML on data that would crash your laptop instantly. Today's task, crime prediction using the Chicago Crimes dataset, is a perfect example. The full dataset has millions of records, and in production a city would be ingesting new crimes every minute. This is why we need distributed ML. It is not a nice-to-have, it is a necessity.

What Breaks When You Try scikit-learn on Big Data



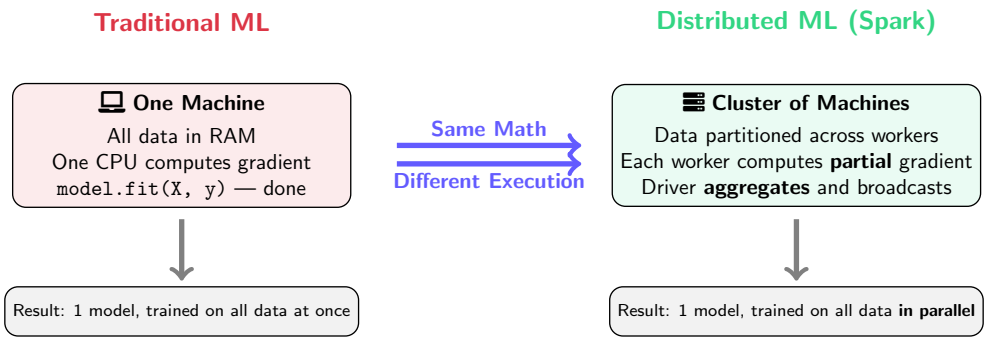
The Root Cause — Three Walls

Memory wall: scikit-learn loads **all data into one machine's RAM**.
Compute wall: Trains on **one CPU** (or a few cores at best).
Scale wall: There is **no way to add more machines** — you're stuck.

Instructor Notes

Now let me show you exactly what goes wrong. Look at the three scenarios on this slide. The first one, you try to load a 150-gigabyte CSV with pandas and you get a MemoryError immediately — the data does not even fit in RAM. The second one, maybe the data does fit, but `model.fit` runs on a single CPU thread for eight hours. You are sitting there waiting, watching your fan spin. The third one is the worst — GridSearchCV with 50 parameter combinations and 10-fold cross-validation. That is 500 model fits, and on one machine it could take four days. Now look at the three walls at the bottom: memory, compute, scale. The fundamental problem is that scikit-learn was designed for one machine. There is no way to say "use ten machines instead." That is the gap Spark MLlib fills.

Traditional ML vs Distributed ML — What Actually Changes



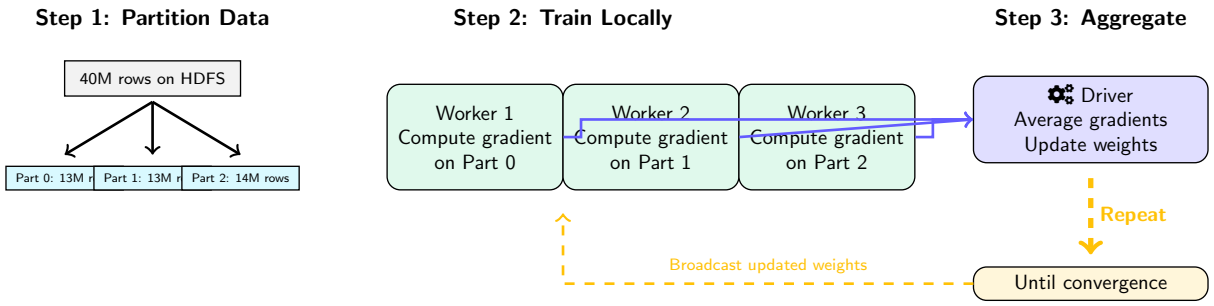
The Fundamental Shift

The **algorithm** (e.g., gradient descent) is the same. The **execution** changes: data is split, gradients are computed locally, then **aggregated** on the driver. You don't learn new math — you learn a new **execution model**.

Instructor Notes

This is the most important conceptual slide of the day, so take a good look. On the left, traditional ML — one machine, all data in RAM, one CPU crunching through gradient descent. On the right, distributed ML — a cluster of machines, data partitioned, each worker computing a partial gradient, and the driver aggregating the results. Now see those two arrows in the middle? Same math, different execution. That is the entire message of today's lecture in two phrases. You do not need to learn new algorithms. Gradient descent is still gradient descent. Random Forest is still Random Forest. What changes is that the data is split across machines and the computation happens in parallel. The end result at the bottom is identical — one trained model. The path to get there is what differs.

How Spark Actually Trains a Model



Why This Is Faster

3 workers each process 13M rows = **3**× **faster** than 1 machine on 40M rows. Add more workers → near-linear speedup. **Same math, parallel execution.**

Instructor Notes

Let me walk you through this diagram step by step because this is how Spark trains a model under the hood. Step 1 on the left: you have 40 million rows on HDFS, and Spark partitions them across three workers — roughly 13 million each. Step 2 in the middle: each worker independently computes a gradient on its local partition. This is the exact same gradient computation you learned in your ML course, just happening three times in parallel on smaller chunks. Step 3 on the right: the driver collects these partial gradients, averages them, and updates the model weights. Then see the dashed arrow going back? The updated weights get broadcast back to the workers, and we repeat until convergence. Three workers, three times faster. Add ten workers, ten times faster. That is the power of distributed training.

Spark MLlib — Same Idea, Scales Across the Cluster

scikit-learn (1 machine)

```
import pandas as pd
from sklearn.ensemble import (
    RandomForestClassifier)
from sklearn.model_selection import (
    train_test_split)

df = pd.read_csv("crimes.csv")
X = df[["District","Hour"]]
y = df["Arrest"]
X_tr, X_te, y_tr, y_te = (
    train_test_split(X, y, test_size=0.2))

model = RandomForestClassifier()
model.fit(X_tr, y_tr)
```

✗ Crashes on large data

Spark MLlib (cluster)

```
from pyspark.ml.classification import (
    RandomForestClassifier)
from pyspark.ml import Pipeline

df = spark.read.csv("hdfs:///...")
train_df, test_df = (
    df.randomSplit([0.8, 0.2]))

rf = RandomForestClassifier(
    featuresCol="features",
    labelCol="label")
pipeline = Pipeline(
    stages=[indexer, assembler, rf])

model = pipeline.fit(train_df)
```

✓ Runs on terabytes, any cluster

The Key Difference

scikit-learn: data is a numpy array in RAM → one machine does everything.
MLlib: data is a distributed DataFrame on HDFS → cluster works in parallel.

Instructor Notes

Now let us look at actual code side by side. On the left, scikit-learn — you load with pandas, split with `train_test_split`, create a classifier, and call `fit`. On the right, Spark MLlib — you load with `spark.read`, split with `randomSplit`, create a classifier and a pipeline, and call `pipeline.fit`. Look how similar the structure is. The main differences are: first, Spark requires a Pipeline to chain steps together. Second, you specify `featuresCol` and `labelCol` explicitly. Third, the data source is HDFS, not a local file. But the logic is identical: load, split, train, predict. If you can write the left column, you can write the right column. The API names change slightly, but the mental model is the same.

Your scikit-learn Knowledge → MLlib Translation Guide

Concept	scikit-learn	Spark MLlib
Load data	`pd.read_csv()`	`spark.read.csv()`
Data structure	`DataFrame` (pandas)	`DataFrame` (Spark)
Feature vector	numpy array / column list	`VectorAssembler`
Encode categories	`LabelEncoder`	`StringIndexer`
One-hot encoding	`OneHotEncoder`	`OneHotEncoder`
Train/test split	`train_test_split()`	`df.randomSplit()`
Chain steps	`sklearn.pipeline`	`pyspark.ml.Pipeline`
Train	`model.fit(X, y)`	`pipeline.fit(df)`
Predict	`model.predict(X)`	`model.transform(df)`
Grid search	`GridSearchCV`	`CrossValidator`
Save model	`joblib.dump()`	`model.save("hdfs://...")`

Notice the Pattern

The **concepts are identical**. The API names are slightly different, and the data lives on a **cluster instead of RAM**.

Instructor Notes

This table is your cheat sheet — I would recommend taking a screenshot of this one. Every row maps a concept you already know from scikit-learn to its Spark MLlib equivalent. Let me highlight a few interesting ones. Look at the feature vector row: in scikit-learn you just pass a list of columns. In Spark, you need a `VectorAssembler` to combine columns into a single vector. That is one extra step you have to learn. Now look at predict: in scikit-learn it is `model.predict()`, in Spark it is `model.transform()`. Why transform? Because Spark adds a prediction column to the DataFrame rather than returning a separate array. And save — instead of `joblib.dump()` to a local file, Spark saves to HDFS so any node in the cluster can load it. Keep this table handy during the lab session.

When to Use scikit-learn vs Spark MLlib

Use scikit-learn

- ✔ Data fits in RAM (<5–10 GB)
- ✔ Rapid prototyping / exploration
- ✔ Need exotic algorithms (XGBoost, SVM kernels, DBSCAN, etc.)
- ✔ Small team, quick iteration

Use Spark MLlib

- ✔ Data is too large for one machine
- ✔ Data already lives on HDFS/S3
- ✔ Need to train on 10 GB+ regularly
- ✔ Production pipeline at scale
- ✔ Need parallel hyperparameter tuning

Decision: data size + infrastructure

Common Mistake

Distributed $\neq$ always better. Spark has **overhead** (serialization, shuffle, coordination). On small data, scikit-learn is **faster**. Use the right tool for the right scale.

Instructor Notes

Now, this is a question I get every year: should I always use Spark? The answer is absolutely not. Look at the decision box at the bottom — it comes down to data size and infrastructure. If your data fits in RAM, say under five to ten gigabytes, scikit-learn is actually faster because there is zero overhead from serialization, shuffling, and network communication. Also notice the left column mentions exotic algorithms — Spark MLlib does not have everything. No SVM kernels, no DBSCAN, no XGBoost natively. So if you need those, stick with scikit-learn or specialized libraries. The right column is when your data is genuinely too large, it already lives on HDFS or S3, and you need production-grade pipelines. Think of it as choosing between a kitchen knife and an industrial machine — both cut, but you would not use the industrial machine for a single onion.

Learning Objectives

By the end of this session you will be able to:

1. Explain **why** traditional ML breaks at scale and how distributed ML solves it
2. Describe the ML Pipeline API: `Transformer`, `Estimator`, `Pipeline`
3. Prepare features with `StringIndexer` and `VectorAssembler`
4. Train classification models (Logistic Regression, Random Forest, GBT)
5. Evaluate with AUC-ROC, accuracy, F1, precision, recall
6. Tune hyperparameters with `CrossValidator`
7. Interpret feature importances to understand model decisions

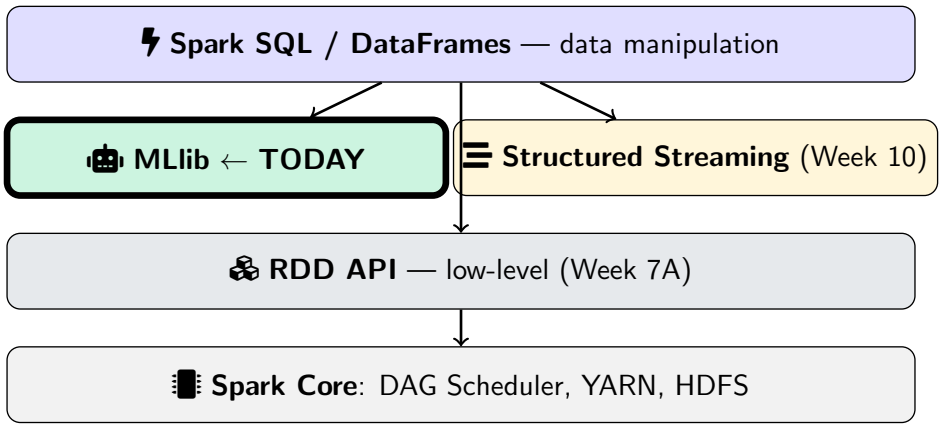
Today's Task

Build an ML pipeline to predict whether a crime results in **arrest** using the Chicago Crimes dataset — the same dataset you've been analyzing all semester.

Instructor Notes

Let me walk through these seven objectives because each one maps to a section of today's lecture. Objective one, we just covered — why scikit-learn breaks and how distributed training fixes it. Objective two is about the Pipeline API, which is the organizational backbone of Spark ML. Objectives three through four are the hands-on core: feature engineering and model training. Objective five, evaluation metrics — same ones you know, just computed differently. Objective six, hyperparameter tuning with CrossValidator, which is Spark's version of GridSearchCV. And objective seven, feature importances, which is how you explain your model's decisions. Notice at the bottom: our task is predicting arrest from the Chicago Crimes dataset. You have been querying this data with SQL and Spark DataFrame operations all semester. Now you are going to predict outcomes from it.

MLlib in the Spark Ecosystem



API Warning

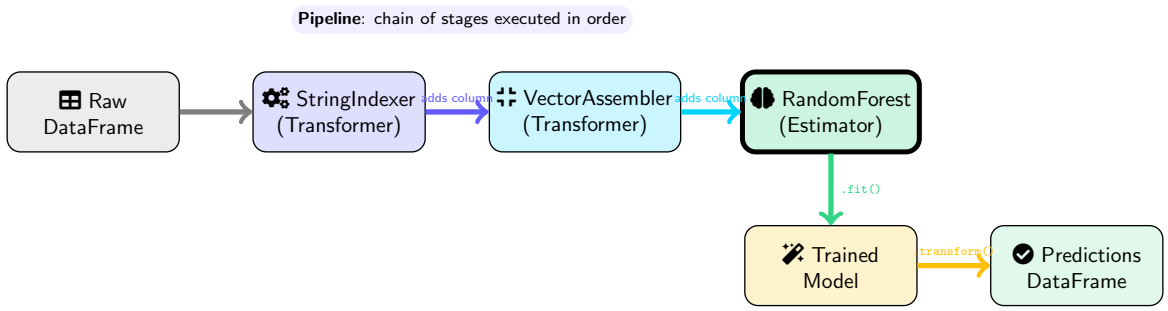
`pyspark.mllib` (RDD-based) — **old, maintenance mode, do NOT use**.
`pyspark.ml` (DataFrame-based) — **current, recommended**. We use this.

Instructor Notes

Take a look at where MLlib sits in the Spark ecosystem. See how it sits right next to Structured Streaming, both built on top of Spark SQL and DataFrames? That means everything you learned in Week 7 about DataFrames — `select`, `filter`, `groupBy` — is directly used by MLlib. Your data goes from DataFrame operations into ML pipelines seamlessly. Now, pay attention to the warning at the bottom. There are two ML packages in Spark: `pyspark.mllib` with a lowercase L, which is the old RDD-based API, and `pyspark.ml`, which is the current DataFrame-based API. If you Google tutorials, some will use the old one. Do not use `mllib` — it is in maintenance mode and will eventually be removed. We use `pyspark.ml` exclusively.

Outline

Pipeline Concept — Assembly Line



Transformer

Takes a DF, returns a DF
Method: `.transform(df)`
Example: VectorAssembler

Estimator

Takes a DF, **learns** params
Method: `.fit(df)` → Model
Example: RandomForest

Instructor Notes

Think of a pipeline like a factory assembly line. See the arrows on this diagram? Raw DataFrame goes in on the left. It flows through StringIndexer, which adds a column. Then through VectorAssembler, which adds another column. Then into RandomForest, which learns from the data. The output is a trained model, and when you call `transform` on new data, you get predictions. Now look at the two boxes at the bottom. A Transformer takes a DataFrame and returns a modified DataFrame — it does not learn anything. An Estimator takes a DataFrame and learns parameters from it via `.fit()`, producing a model. This Transformer-Estimator distinction is the same as in scikit-learn. If you have used `sklearn.pipeline.Pipeline`, this is the exact same idea, just with slightly different names.

Transformer vs Estimator — Detailed Comparison

	Transformer	**Estimator**
Method	`.transform(df)` → df	`.fit(df)` → Model (Transformer)
Parameters	Fixed (no learning)	Learned from data
Examples	VectorAssembler, OneHotEncoder (fitted), StringIndexer (fitted)	LogisticRegression, RandomForest, StringIndexer (unfitted)
sklearn analogy	`.transform()` after `.fit()`	`.fit()` itself
Analogy	A **ruler** (measures directly)	A **student** (learns, then measures)

Important Nuance

`StringIndexer` starts as an **Estimator** (learns category → index mapping via `.fit()`), then produces a fitted **Transformer** (applies the mapping via `.transform()`). Same as `sklearn.preprocessing.LabelEncoder` — `.fit()` then `.transform()`.

Instructor Notes

Now let us look at this comparison more carefully. The analogy row is my favorite: a Transformer is like a ruler — it just measures, no learning needed. An Estimator is like a student — it has to study the data first, then it can make measurements. Now, here is the nuance that trips people up — look at the alert box at the bottom. StringIndexer is an Estimator when unfitted because it needs to learn which categories exist and assign indices. But after you call `.fit()`, it produces a fitted Transformer that can apply the learned mapping. This is exactly how scikit-learn's LabelEncoder works: `fit()` learns the mapping, `transform()` applies it. Inside a Pipeline, Spark handles this automatically — you do not need to call fit and transform separately. The pipeline does it for you.

Why Pipelines Matter **More** in Distributed ML

✗ Without Pipeline (danger zone)

- Fit StringIndexer on full data → **data leakage**
- Manually apply each step → **inconsistent** train vs test
- Can't serialize to HDFS → **can't deploy** to cluster
- Each worker needs transforms → **manual coordination**

✓ With Pipeline

- `pipeline.fit(train_df)` → fits **only on train**
- `model.transform(test_df)` → **same steps** automatically
- `model.save("hdfs://...")` → **any worker** can load
- Pipeline ships transforms to every worker **automatically**

```
pipeline = Pipeline(stages=[
    string_indexer,        # Stage 1: encode categories
    vector_assembler,      # Stage 2: assemble features
    classifier             # Stage 3: train model
])
model = pipeline.fit(train_df)        # fit everything
predictions = model.transform(test_df) # apply everything
```

In scikit-learn you *can* skip pipelines. In Spark, you shouldn't.

Distributed data across nodes means transforms must travel **with** the model.

Instructor Notes

This is where it gets really interesting. In scikit-learn, pipelines are a nice convenience but you can get away without them. In Spark, they are practically mandatory. Look at the left column — without a pipeline, you risk data leakage because you might accidentally fit a StringIndexer on the full dataset including test data. You also cannot serialize individual steps to HDFS easily, which means you cannot deploy your model. Now look at the right column. With a pipeline, `pipeline.fit(train_df)` guarantees that every stage is fit only on training data. `model.transform(test_df)` applies the exact same transformations automatically. And you can save the entire thing to HDFS with one line. The key insight at the bottom is critical: in a distributed setting, the transformations must travel with the model to every worker node. A pipeline packages everything together.

Outline

Our Dataset: Chicago Crimes

Column	Type	Role in ML
District	int	✔ Feature (numeric)
Primary Type	string	✔ Feature (categorical → encode)
Hour (extracted)	int	✔ Feature (numeric)
Domestic	boolean	✔ Feature (categorical → encode)
Arrest	**boolean**	🏁 **Label (what we predict)**

🎯 **Goal**: Given District, Crime Type, Hour, Domestic → predict **Arrest** (True/False)

Instructor Notes

Alright, let us talk about our dataset. You have been working with Chicago Crimes all semester, so the columns should be familiar. But now we are looking at them through an ML lens. See the table here — District and Hour are numeric features we can use directly. Primary Type and Domestic are categorical, so they need encoding before we can feed them to a model. And look at the highlighted row: Arrest is our label, the thing we want to predict. This is a binary classification problem — given information about a crime, can we predict whether it leads to an arrest? True or False. Notice this is exactly the kind of problem you solved in your ML course, just with a dataset you already understand deeply from your Spark SQL and DataFrame work earlier in the semester.

Why These Features? Think Before You Code

In your ML course you learned: feature selection should be driven by **domain knowledge**, *not guessing.*

Feature	Domain Intuition	Expected Effect
Primary Type	NARCOTICS → almost always arrest. THEFT → rarely catch in act.	Strong predictor — crime type determines police procedure
Domestic	Many jurisdictions have **mandatory arrest** policies for domestic crimes	Strong predictor — policy-driven
District	Some districts have more resources, higher patrol density	Moderate predictor — varies by staffing
Hour	Crimes at 2 AM: fewer witnesses, harder to catch. vs 2 PM: more police visible	Moderate predictor — time affects opportunity

What We're NOT Using (and Why)

Block (address) → too many unique values, would overfit. **Date** → raw string, but we extract **Hour** as a useful signal. **Year** → could capture trend but risks data leakage on temporal data.

Instructor Notes

Before we write a single line of code, let us think about why we are choosing these features. This is something you learned in your ML course — feature selection should be driven by domain knowledge, not by throwing everything at the model and hoping for the best. Look at the table. Primary Type is our strongest expected predictor. Why? Because if someone is caught with narcotics, the arrest happens on the spot. Theft, on the other hand, the criminal is usually long gone. Domestic is strong because of mandatory arrest policies — the law requires an arrest in many domestic cases. District and Hour are moderate because they capture environmental factors like police staffing and time of day. Now look at what we are NOT using and why. Block has too many unique values and would cause overfitting. Year could leak temporal information. This kind of reasoning is what separates good data scientists from people who just run code.

Step 1: Extract Features from Raw Data

In scikit-learn: `df["Hour"] = pd.to_datetime(df["Date"]).dt.hour`. In Spark:

```
from pyspark.sql.functions import hour, to_timestamp, col

# Load dataset from HDFS (distributed!)
df = spark.read.csv("hdfs:///data/chicago_crimes.csv",
                    header=True, inferSchema=True)

# Extract hour of day from Date string
df = df.withColumn("Hour",
    hour(to_timestamp(col("Date"),
         "MM/dd/yyyy hh:mm:ss a")))

# Select relevant columns, drop nulls
df = df.select("District", "Primary Type",
               "Hour", "Domestic", "Arrest").dropna()

# Cast label to integer (True=1, False=0)
df = df.withColumn("label",
    col("Arrest").cast("integer"))
```

Tip

Always check for nulls with `df.filter(col("Hour").isNull()).count()` before training. Nulls silently break ML models.

Instructor Notes

Now let us start coding. This is Step 1 — extracting features from the raw data. Look at the scikit-learn equivalent at the top: `pd.to_datetime(df["Date"]).dt.hour`. In Spark, we use `to_timestamp` combined with `hour` to do the same thing. The key line is the `withColumn("Hour", ...)` call, which parses the date string and extracts the hour component. Notice the date format string `"MM/dd/yyyy hh:mm:ss a"` — the "a" at the end handles AM/PM. Then we select only the columns we need and drop nulls. The last line casts the Arrest boolean to an integer because Spark ML models expect numeric labels: 1 for arrest, 0 for no arrest. The tip at the bottom is important — always check for nulls before training. A single null can silently corrupt your model or cause cryptic errors.

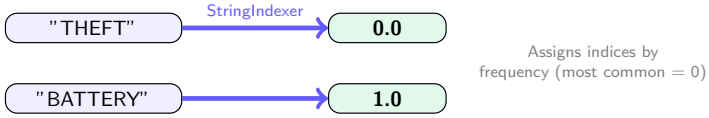
Step 2: StringIndexer — Encoding Categories

In scikit-learn: `LabelEncoder().fit_transform(df["Primary Type"])`. In Spark:

```python
from pyspark.ml.feature import StringIndexer

# "THEFT" -> 0.0, "BATTERY" -> 1.0, ...
crime_indexer = StringIndexer(
    inputCol="Primary Type",
    outputCol="crime_index",
    handleInvalid="skip"    # drop rows with unseen categories
)

# "false" -> 0.0, "true" -> 1.0
domestic_indexer = StringIndexer(
    inputCol="Domestic",
    outputCol="domestic_index",
    handleInvalid="skip"
)
```



Instructor Notes

Step 2 is encoding categorical columns. Look at the diagram at the bottom — StringIndexer converts string values to numeric indices. "THEFT" becomes 0.0 because it is the most frequent crime type. "BATTERY" becomes 1.0, and so on. This is exactly what scikit-learn's LabelEncoder does. There are two things to notice in the code. First, each StringIndexer handles one column, so we need two: one for Primary Type and one for Domestic. Second, see the `handleInvalid="skip"` parameter? That tells Spark to drop rows with categories it has never seen during training. This matters in production when new crime types might appear in future data. Without this setting, your model would crash on unseen categories. These indexer objects are not fitted yet — they will be fitted automatically when we call `pipeline.fit()` later.

StringIndexer vs OneHotEncoder — When to Use Which

StringIndexer Only

"THEFT" → **0.0**
"BATTERY" → **1.0**
"ROBBERY" → **2.0**

StringIndexer + OneHotEncoder

"THEFT" → **(1, 0, 0)**
"BATTERY" → **(0, 1, 0)**
"ROBBERY" → **(0, 0, 1)**

	StringIndexer only	**+ OneHotEncoder**
Use with	✔ Tree models (RF, GBT)	✔ Linear models (LR)
Why?	Trees split on values, no order assumed	LR treats 2.0 as "more" than 1.0 → wrong
Feature count	1 column per category	$N - 1$ columns (sparse)

Rule of Thumb (Same as scikit-learn!)

Tree-based models → StringIndexer is enough. **Linear models** → add OneHotEncoder to avoid false ordinal relationships.

Instructor Notes

This is a question that comes up constantly: when do I need OneHotEncoder on top of StringIndexer? Look at the two encoding schemes side by side. On the left, StringIndexer alone assigns 0, 1, 2 to crime types. On the right, OneHotEncoder expands that into binary vectors. Why does this matter? Because of the table in the middle. Tree models like Random Forest and GBT split on threshold values, so they naturally handle the indices without assuming any ordering. But Logistic Regression is a linear model — it sees 2.0 as numerically "more" than 1.0, which implies ROBBERY is somehow "greater" than BATTERY. That is obviously wrong. So the rule of thumb at the bottom, which is the same rule you learned in scikit-learn, is: trees are fine with just indexing, linear models need one-hot encoding. For today's lab, since we are using Random Forest as our main model, StringIndexer alone will work.

Step 3: VectorAssembler — Combine into Feature Vector

In scikit-learn: `X = df[["District","crime_index","Hour","domestic_index"]]`. In Spark:

```
from pyspark.ml.feature import VectorAssembler

assembler = VectorAssembler(
    inputCols=["District", "crime_index",
               "Hour", "domestic_index"],
    outputCol="features"
)
```

District	crime_index	Hour	domestic_idx		features
8	0.0	14	1.0	→	[8, 0, 14, 1]
11	1.0	22	0.0		[11, 1, 22, 0]

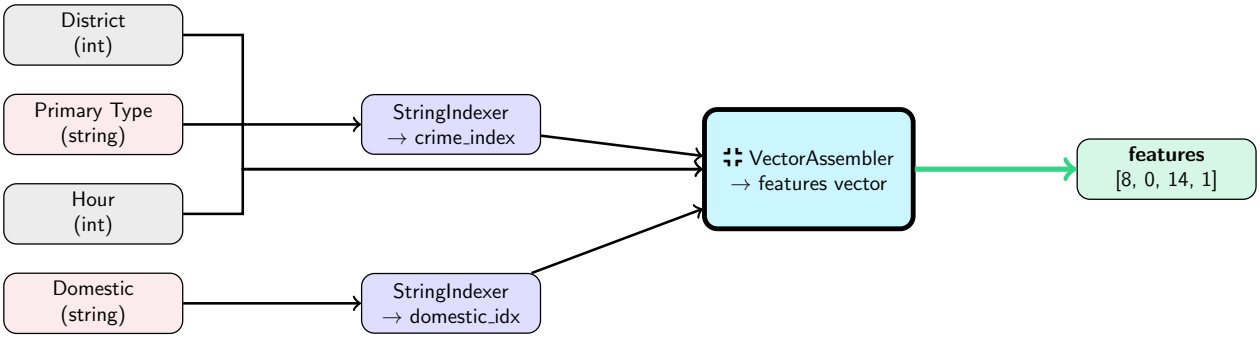
Why a Single Vector?

All Spark ML algorithms require input as a **single vector column** called `features`. This is different from scikit-learn where you pass a matrix. VectorAssembler creates a `DenseVector` per row.

Instructor Notes

This step is unique to Spark and does not have a direct equivalent in scikit-learn. Look at the table in the diagram. We have four separate columns: District, crime_index, Hour, and domestic_index. VectorAssembler takes all of these and packs them into a single column called "features" where each row is a vector like [8, 0, 14, 1]. Why does Spark require this? Because all Spark ML algorithms expect exactly one input column called "features" containing a vector. In scikit-learn, you just pass a matrix or a list of columns. In Spark, you must explicitly assemble columns into a vector first. It is one extra step, but it has a benefit: it makes the pipeline explicit about which columns are features. Notice in the code, we list the input columns by name — this is self-documenting and prevents accidental feature inclusion.

Feature Engineering Summary



Pipeline Order Matters

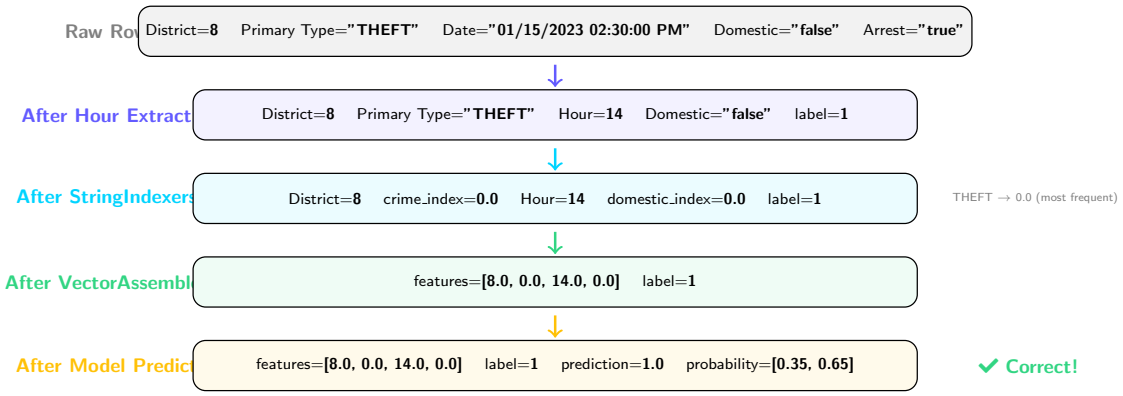
StringIndexers must run **before** VectorAssembler, because the assembler needs the numeric index columns to exist. Same logic as scikit-learn's `ColumnTransformer` ordering.

Instructor Notes

Let me give you the big picture of feature engineering with this diagram. Follow the arrows from left to right. On the far left, we have our raw columns. District and Hour are already numeric, so they go straight to the VectorAssembler — see the arrows bypassing the StringIndexers. Primary Type and Domestic are strings, so they must pass through StringIndexers first to become numeric indices. Then everything converges at the VectorAssembler in the middle, which produces the final features vector on the right. The note at the bottom is important: order matters. If you try to run VectorAssembler before StringIndexer, it will fail because the `crime_index` column does not exist yet. This is exactly like scikit-learn's ColumnTransformer — you have to encode before you assemble. The pipeline handles this ordering automatically when you list stages in the correct sequence.

One Row Through the Pipeline — Step by Step

Let's trace a single Chicago crime record through every stage:



Every Row Goes Through This

The pipeline applies these 4 stages to **every row in parallel across the cluster**. On 40M rows with 4 workers, each worker processes ∼10M rows simultaneously.

Instructor Notes

This is one of my favorite slides because it makes the abstract concrete. Let us trace one actual crime record through every stage. At the top, the raw row: District 8, THEFT, a date string, not domestic, arrested. After hour extraction, the date becomes Hour 14 (that is 2:30 PM) and Arrest becomes label 1. After StringIndexers, "THEFT" becomes 0.0 because it is the most frequent crime type, and "false" for Domestic becomes 0.0. After VectorAssembler, everything is packed into features = [8.0, 0.0, 14.0, 0.0]. And after the model predicts, we get prediction 1.0 with probability [0.35, 0.65], meaning the model is 65 percent confident this crime leads to an arrest — and it was correct. Now multiply this by 40 million rows running in parallel across four workers. That is the power of the pipeline.

Outline

Train/Test Split

In scikit-learn: `train_test_split(X, y, test_size=0.2)`. In Spark:

```
# 80% train, 20% test
train_df, test_df = df.randomSplit([0.8, 0.2], seed=42)

print(f"Training: {train_df.count()} rows")
print(f"Testing:  {test_df.count()} rows")

# IMPORTANT: Check class balance!
train_df.groupBy("label").count().show()
```

Class Imbalance Warning

If 85% of crimes have `Arrest=False`, a model can just predict False and get 85% accuracy! $\rightarrow$ Always check F1 and AUC, not just accuracy.

seed=42

Fixed seed ensures **reproducible** splits — same rows in train/test every time you run. Same concept as `random_state=42` in sklearn.

Instructor Notes

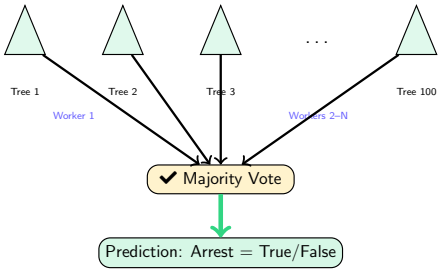
So now that you understand feature engineering, let us talk about actually training models. Before we train anything, we need to split the data. Look at the code — `randomSplit([0.8, 0.2], seed=42)` gives us 80 percent for training and 20 percent for testing. The `seed=42` is the same concept as `random_state=42` in scikit-learn — it ensures reproducible results. Now, look at the class imbalance warning on the left. This is critical for our dataset. About 85 percent of crimes in Chicago do not lead to arrest. That means a lazy model that always predicts "No Arrest" would get 85 percent accuracy and catch zero criminals. That is why we will use F1 score and AUC-ROC as our primary metrics, not accuracy. Always check `groupBy("label").count()` before training to understand the class distribution.

Random Forest Classifier

Same algorithm you know from your ML course — but Spark trains trees **in parallel across workers**.

```
from pyspark.ml.classification import RandomForestClassifier

rf = RandomForestClassifier(
    featuresCol="features",
    labelCol="label",
    numTrees=100,      # 100 decision trees
    maxDepth=5,        # max depth per tree
    seed=42
)
```



Spark distributes tree training across workers — each worker builds subset of trees in parallel

Instructor Notes

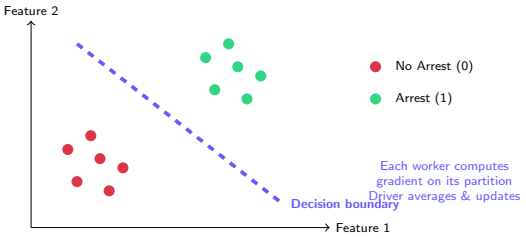
Random Forest — you know this algorithm well. Build many decision trees on random subsets of data and features, then take a majority vote. What is different in Spark? Look at the diagram — see how trees are distributed across workers? Worker 1 might build trees 1 through 25, Worker 2 builds trees 26 through 50, and so on. Since each tree in a Random Forest is independent, this parallelizes beautifully. In the code, notice `featuresCol="features"` and `labelCol="label"` — you must tell Spark which columns to use, unlike scikit-learn where you pass X and y directly. `numTrees=100` means 100 trees, and `maxDepth=5` limits each tree's depth to prevent overfitting. These are the same hyperparameters you tuned in scikit-learn. The algorithm is identical; only the execution is distributed.

Logistic Regression

Same sigmoid function — but gradient descent runs **across workers**, *each computing partial gradients.*

```python
from pyspark.ml.classification import LogisticRegression

lr = LogisticRegression(
    featuresCol="features",
    labelCol="label",
    maxIter=100,       # max gradient descent steps
    regParam=0.01      # L2 regularization strength
)
```



When to Use

Fast baseline model. Works best for linearly separable data. Always start here, then try Random Forest or GBT.

Instructor Notes

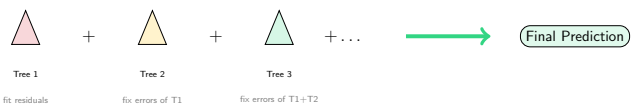
Logistic Regression — your simplest and fastest classifier. Look at the diagram: red dots are no-arrest cases, green dots are arrest cases, and the dashed line is the decision boundary that the sigmoid function learns. The distributed aspect is shown in the note on the right: each worker computes the gradient on its own partition of data, then the driver averages those gradients and updates the weights. This is exactly the partition-compute-aggregate pattern we saw earlier. In the code, `maxIter=100` controls how many gradient descent steps to take, and `regParam=0.01` is L2 regularization to prevent overfitting — same as scikit-learn's `C` parameter but inverted. My recommendation, which you will also see in industry, is always start with Logistic Regression as your baseline. If it performs well enough, you are done. If not, move to Random Forest or GBT.

Gradient-Boosted Trees (GBT) — Often the Best Performer

You may know XGBoost from your ML course. GBTClassifier is Spark's distributed equivalent.

```python
from pyspark.ml.classification import GBTClassifier

gbt = GBTClassifier(
    featuresCol="features",
    labelCol="label",
    maxIter=50,        # number of boosting rounds
    maxDepth=5,        # depth per tree
    stepSize=0.1,      # learning rate
    seed=42
)
```



RF: Trees are independent (bagging) → easy to parallelize
GBT: Trees are **sequential** (boosting) → each corrects previous errors
GBT often wins on accuracy, RF is faster to train

GBT Limitation in Spark

Spark's GBTClassifier currently only supports **binary classification**. For multiclass, use Random Forest or Logistic Regression.

Instructor Notes

Now, this is where it gets interesting — Gradient-Boosted Trees. If you have used XGBoost in your ML course, GBT is Spark's distributed version. Look at the diagram carefully. Unlike Random Forest where trees are independent, in GBT each tree corrects the errors of the previous one. Tree 1 fits the data, Tree 2 fixes what Tree 1 got wrong, Tree 3 fixes what Tree 1 plus Tree 2 still missed, and so on. The `stepSize=0.1` is the learning rate, which controls how aggressively each tree corrects errors. The key trade-off is at the bottom: because trees are sequential, GBT is harder to parallelize than Random Forest. But it often achieves the best accuracy. Also note the limitation in the alert box — Spark's GBTClassifier only supports binary classification. Our arrest prediction is binary, so we are fine. But if you had three or more classes, you would need Random Forest or Logistic Regression instead.

Complete Pipeline — Putting It Together

```
from pyspark.ml import Pipeline

pipeline = Pipeline(stages=[
    crime_indexer,        # "Primary Type" -> crime_index
    domestic_indexer,     # "Domestic" -> domestic_index
    assembler,            # columns -> features vector
    rf                    # RandomForestClassifier
])

# Cache training data (iterative training benefits!)
train_df.cache()

# Train the entire pipeline
model = pipeline.fit(train_df)

# Predict on test data
predictions = model.transform(test_df)
predictions.select("features", "label",
                   "prediction", "probability").show(5)
```

Key Principle

`pipeline.fit(train_df)` fits **all stages** in order. `model.transform(test_df)` applies **all stages** in order. Compare with sklearn: `pipe.fit(X_train)` then `pipe.predict(X_test)` — same idea!

Instructor Notes

Here is the moment where everything comes together. Look at the Pipeline stages list: first the two StringIndexers, then the VectorAssembler, then the Random Forest classifier. Four stages, in order. When you call `pipeline.fit(train_df)`, Spark runs through all four stages sequentially on the training data — encoding, assembling, and training. The result is a fitted model that encapsulates every transformation. Then `model.transform(test_df)` applies all four stages to the test data automatically. Notice the `train_df.cache()` call before fitting. Remember from Week 8A, ML algorithms are iterative — Random Forest reads the data multiple times to build trees. Caching keeps the training data in memory instead of re-reading from HDFS each time. This can cut training time significantly. The alert box at the bottom draws the parallel to scikit-learn: `pipe.fit()` then `pipe.predict()` — same concept, same flow.

Outline

What Predictions Look Like

features	label	prediction	probability	
[8, 0, 14, 1]	0	0.0	[0.82, 0.18]	✓
[11, 1, 22, 0]	1	1.0	[0.35, 0.65]	✓
[3, 2, 3, 1]	1	0.0	[0.55, 0.45]	✗

prediction

Model's binary answer: 0.0 or 1.0
(sklearn: `model.predict()`)

probability

Confidence: [P(class=0), P(class=1)]
(sklearn: `model.predict_proba()`)

Instructor Notes

Let us look at what comes out of the model after prediction. See the table here? Each row shows features, the true label, the model's prediction, and the probability. Look at the first row — the model predicted 0.0 (no arrest) with 82 percent confidence, and the true label was 0. Correct. Second row, predicted 1.0 (arrest) with 65 percent confidence, true label was 1. Also correct. But the third row is interesting — the model predicted 0.0 but the true label was 1. The model missed an arrest, and look at the probability: [0.55, 0.45]. The model was barely confident, almost a coin flip. This is where evaluation metrics come in — we need to quantify how often the model gets it right and wrong. Notice that unlike scikit-learn where `predict` and `predict_proba` are separate calls, Spark gives you both in the same output DataFrame.

Evaluation Metrics — You Know These!

Same metrics from your ML course. The formulas don't change in distributed ML.

Metric	Formula	Meaning for Our Task
Accuracy	$\frac{TP+TN}{\text{Total}}$	% of correct arrest predictions
Precision	$\frac{TP}{TP+FP}$	Of predicted arrests, how many were real?
Recall	$\frac{TP}{TP+FN}$	Of real arrests, how many did we catch?
F1 Score	$\frac{2 \cdot P \cdot R}{P+R}$	Balance between P and R
AUC-ROC	Area under ROC curve	Overall ranking quality (0.5 = random)

Why Not Just Accuracy? (Refresher)

Chicago crimes: ∼85% have No Arrest. A model predicting "No Arrest" for everything gets **85% accuracy** but catches **zero criminals**. Always check **F1** and **AUC**.

Distributed Metric Computation

Spark computes TP/TN/FP/FN counts **per partition**, then aggregates across workers. Same formulas, parallel counting.

Instructor Notes

These metrics should look very familiar from your ML course. Accuracy, Precision, Recall, F1, AUC — the formulas are exactly the same. What I want to emphasize is the third column: what these metrics mean for our specific task. Precision tells us: of all the times the model said "arrest," how many were actually arrested? A low precision means wasted police resources chasing false alarms. Recall tells us: of all actual arrests, how many did we catch? A low recall means criminals walking free. The alert box at the bottom is crucial — with 85 percent of crimes having no arrest, accuracy is misleading. A model that always says "no arrest" is 85 percent accurate but completely useless. That is why F1 and AUC are our real metrics. And notice the distributed aspect: Spark counts TP, TN, FP, FN per partition in parallel, then aggregates. Same math, parallel execution.

Confusion Matrix — Visual Guide

Actual \ Predicted	No Arrest (0)	Arrest (1)
No Arrest (0)	**TN** 6800	**FP** 350
Arrest (1)	**FN** 420	**TP** 1430

TN: Correctly said No Arrest ✔

FP: False alarm — wasted resources ✘

FN: Missed arrest — criminal walks free ✘

TP: Correctly predicted arrest ✔

Reading the Matrix

$\text{Precision} = \frac{1430}{1430+350} = 0.80$ $\quad \text{Recall} = \frac{1430}{1430+420} = 0.77$ $\quad \text{F1} = \frac{2\times 0.80\times 0.77}{0.80+0.77} = 0.78$

Instructor Notes

Let us look at the confusion matrix with concrete numbers from our crime prediction task. See the four quadrants? The green cells are where the model is correct: TN means it correctly said no arrest (6800 cases), and TP means it correctly predicted arrest (1430 cases). The red cells are errors: FP is a false alarm where we predicted arrest but there was none (350 cases — wasted police resources). FN is the worst — we predicted no arrest but the person was actually arrested (420 cases — a criminal the model would have let walk free). Now look at the calculations at the bottom. Precision is 0.80, meaning 80 percent of our arrest predictions were correct. Recall is 0.77, meaning we caught 77 percent of actual arrests. F1 at 0.78 balances both. These are exactly the formulas you learned in your ML course, applied to a domain that makes the consequences tangible.

Computing Metrics in Spark

In scikit-learn: `accuracy_score()`, `f1_score()`, etc. In Spark:

```
from pyspark.ml.evaluation import \
    BinaryClassificationEvaluator, \
    MulticlassClassificationEvaluator

# AUC-ROC
binary_eval = BinaryClassificationEvaluator(labelCol="label")
auc = binary_eval.evaluate(predictions)
print(f"AUC-ROC: {auc:.4f}")

# Accuracy, F1, Precision, Recall
mc_eval = MulticlassClassificationEvaluator(
    labelCol="label", predictionCol="prediction")

accuracy = mc_eval.evaluate(predictions,
    {mc_eval.metricName: "accuracy"})
f1 = mc_eval.evaluate(predictions,
    {mc_eval.metricName: "f1"})
precision = mc_eval.evaluate(predictions,
    {mc_eval.metricName: "weightedPrecision"})
recall = mc_eval.evaluate(predictions,
    {mc_eval.metricName: "weightedRecall"})
print(f"Accuracy: {accuracy:.4f}  F1: {f1:.4f}")
print(f"Precision: {precision:.4f}  Recall: {recall:.4f}")

# Confusion matrix
predictions.groupBy("label", "prediction") \
    .count().orderBy("label", "prediction").show()
```

Instructor Notes

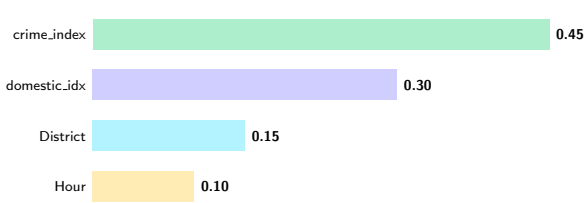
Here is how you actually compute those metrics in Spark code. Notice there are two evaluators: `BinaryClassificationEvaluator` for AUC-ROC, and `MulticlassClassificationEvaluator` for accuracy, F1, precision, and recall. Why two? Because AUC-ROC uses probabilities and is specific to binary classification, while accuracy and F1 work with the prediction column and apply to any number of classes. The pattern is straightforward: create an evaluator, call `evaluate(predictions)` with the metric you want. For the confusion matrix at the bottom, we use a simple `groupBy` on label and prediction columns — no special library needed. This is one of those slides you will want to copy directly into your lab notebook because you will use this exact code every time you evaluate a model in Spark.

Feature Importances — Which Features Drive Predictions?

Same concept as sklearn's `model.feature_importances_`:

```
# Extract the fitted Random Forest model
rf_model = model.stages[-1]  # last pipeline stage
importances = rf_model.featureImportances

feature_names = ["District", "crime_index",
                 "Hour", "domestic_index"]
for name, imp in zip(feature_names, importances):
    print(f"  {name}: {imp:.4f}")
```



Interpretation — Matches Our Domain Reasoning!

Crime type is the strongest predictor (NARCOTICS → almost always arrest). **Domestic flag** second (mandatory arrest policies). This confirms our domain intuition from the feature selection slide.

Instructor Notes

This is one of the most satisfying parts of any ML project — validating that your model learned what you expected. Look at the bar chart. Crime type index is the strongest predictor at 0.45, followed by the domestic flag at 0.30. Remember our domain reasoning from the feature selection slide? We said crime type would be the strongest because narcotics cases almost always lead to arrest. And domestic crimes have mandatory arrest policies. The model confirmed our intuition. District and Hour are moderate, exactly as we predicted. In the code, notice `model.stages[-1]` — that extracts the last stage of the pipeline, which is the fitted Random Forest model. Then `featureImportances` gives us the importance scores, just like scikit-learn's `feature_importances_` attribute. This kind of interpretability is critical — you should always be able to explain why your model makes the predictions it does.

Outline

Logistic Regression vs Random Forest vs GBT

Aspect	Logistic Reg.	Random Forest	GBT
Training speed	✔ Fastest	Moderate	Slowest (sequential)
Accuracy (typical)	Baseline	Good	✔ Often best
Non-linear	✘ No	✔ Yes	✔ Yes
Interpretability	Coefficients	Feature importances	Feature importances
Overfitting risk	Low	Moderate	⚠ High
Parallelism	✔ Gradient parallel	✔ Trees parallel	⚠ Limited
sklearn equivalent	LogisticRegression	RandomForest	XGBoost / GBM

Recommended Strategy (Same as Traditional ML!)

Step 1: Logistic Regression (fast baseline). **Step 2**: Random Forest (handles non-linearity). **Step 3**: GBT (squeeze out extra accuracy).
If GBT $\gg$ LR, the relationship is non-linear. If LR $\approx$ RF, keep LR (simpler).

Instructor Notes

Let us compare our three classifiers side by side. Look at the table carefully. Logistic Regression is the fastest and has the lowest overfitting risk, but it cannot capture non-linear relationships. Random Forest handles non-linearity and parallelizes beautifully because each tree is independent. GBT often achieves the best accuracy but is the slowest to train and has limited parallelism because trees are sequential. The parallelism row is especially important in a distributed context — Random Forest scales almost linearly with more workers, while GBT does not benefit as much. The strategy at the bottom is the same one you would follow in scikit-learn: start simple with LR, move to RF if you need non-linearity, and try GBT if you need to squeeze out every last percentage point of accuracy. If LR and RF perform similarly, keep LR because simpler models are easier to maintain and explain.

Hyperparameter Tuning — CrossValidator

In scikit-learn: `GridSearchCV`. In Spark: `CrossValidator` + `ParamGridBuilder`.
The big difference? Spark runs folds **in parallel across the cluster**.

```
from pyspark.ml.tuning import CrossValidator, ParamGridBuilder

# Define parameter grid (like sklearn's param_grid)
paramGrid = ParamGridBuilder() \
    .addGrid(rf.numTrees, [50, 100, 200]) \
    .addGrid(rf.maxDepth, [3, 5, 10]) \
    .build()  # 3 x 3 = 9 combinations

# CrossValidator (like sklearn's GridSearchCV)
cv = CrossValidator(
    estimator=pipeline,        # our full pipeline
    estimatorParamMaps=paramGrid,
    evaluator=BinaryClassificationEvaluator(labelCol="label"),
    numFolds=3,                # 3-fold cross-validation
    parallelism=4              # run 4 folds in parallel!
)

# Fit --- Spark tries all 9 combos x 3 folds = 27 models
cvModel = cv.fit(train_df)

# Best model is automatically selected
best_predictions = cvModel.transform(test_df)
```

The Distributed Advantage

`parallelism=4`: Spark trains 4 models **simultaneously** across the cluster. On a laptop, 27 models run sequentially. On a 4-worker cluster, ∼7× faster.

Instructor Notes

This is where the distributed advantage really shines. Look at the code — `ParamGridBuilder` lets you define a grid of hyperparameters, just like scikit-learn's `param_grid` dictionary. We are trying 3 values for numTrees times 3 values for maxDepth, giving us 9 combinations. With 3-fold cross-validation, that is 27 total models to train. In scikit-learn, these 27 models run sequentially on your laptop. In Spark, see the `parallelism=4` parameter? That means Spark trains 4 models simultaneously across the cluster. On a 4-worker cluster, those 27 models finish roughly 7 times faster. The `CrossValidator` automatically selects the best-performing parameter combination, and `cvModel.transform(test_df)` uses that best model for predictions. This is the exact same workflow as GridSearchCV, but with the added benefit of cluster-level parallelism. For your lab, try different grid sizes and see how parallelism affects training time.

ML Best Practices Checklist

1. ✔ **Split before fitting**: Always split data *before* fitting any transformer (prevent data leakage)
2. ✔ **Check class balance**: Use stratified split or class weights if imbalanced
3. ✔ **Use Pipeline**: Ensures consistent transforms on train and test — **critical in distributed settings**
4. ✔ **Multiple metrics**: Evaluate with AUC, F1, precision, recall — not just accuracy
5. ✔ **Cache training data**: `train_df.cache()` before `.fit()` — ML algorithms are iterative, reads data many times
6. ✔ **Tune hyperparameters**: Use `CrossValidator` to find optimal settings — exploits cluster parallelism
7. ✔ **Save model**: `model.save("hdfs:///models/...")` for reuse without retraining

⚠ Data Leakage (Reminder)

If you fit a StringIndexer on the **full dataset** (before splitting), the test set "leaks" information into the model. Always fit inside a Pipeline that sees only `train_df`. *Same rule as scikit-learn — more dangerous at scale because bugs are harder to spot.*

Instructor Notes

Think of this checklist as your pre-flight checklist before submitting any ML job. Let me walk through each one. First, split before fitting — this prevents data leakage, which is the number one silent killer of ML projects. Second, check class balance, which we talked about with the 85 percent no-arrest issue. Third, always use a Pipeline, not manual step-by-step transformations. Fourth, evaluate with multiple metrics, not just accuracy. Fifth, cache your training data because ML training is iterative and reads the data many times. Sixth, tune hyperparameters with CrossValidator to exploit cluster parallelism. And seventh, save your model to HDFS so you do not have to retrain every time. The data leakage warning at the bottom deserves extra emphasis: in a distributed setting, leakage bugs are harder to detect because you cannot easily inspect intermediate data across workers. The Pipeline prevents this automatically.

Saving & Loading Models

In scikit-learn: `joblib.dump(model, "model.pkl")`. In Spark:

```
# Save the trained pipeline model to HDFS
model.save("hdfs:///models/arrest_predictor_rf")

# Later: Load without retraining
from pyspark.ml import PipelineModel
loaded = PipelineModel.load(
    "hdfs:///models/arrest_predictor_rf")

# Use immediately on new data
new_predictions = loaded.transform(new_data)
```

Production Workflow

1. Train pipeline once on cluster (expensive, maybe 30 min)
2. Save model to HDFS — accessible to **any machine** in the cluster
3. Load in production app (cheap, fast, milliseconds)
4. Retrain periodically with new data (weekly/monthly)

Why HDFS Matters

Unlike `joblib.dump()` which saves to **one machine**, Spark saves to **HDFS** — any node in the cluster can load the model. No file copying needed.

Instructor Notes

The last operational piece you need is saving and loading models. Look at the code — `model.save()` writes to HDFS, and `PipelineModel.load()` reads it back. That is it. Two lines. But here is why this matters so much in a distributed setting. When you use `joblib.dump()` in scikit-learn, the model file lives on one specific machine. If another machine needs it, you have to copy the file over. With HDFS, the model is accessible from any node in the cluster immediately. The production workflow at the bottom is what real companies do: train the model once (expensive, might take 30 minutes), save it to HDFS, then load it in a production application where it makes predictions in milliseconds. You retrain periodically — maybe weekly or monthly — when you have fresh data. For your milestone project, saving and loading means you can train once and demonstrate predictions without waiting for retraining every time.

Outline

Key Takeaways

1. **Same math, different execution**: ML algorithms don't change — the **data distribution** and **parallel training** are what's new
2. **Spark MLlib** = distributed ML on DataFrames (`pyspark.ml`)
3. **Pipeline API**: Transformer → Estimator → Model — like sklearn pipelines, but essential for distributed consistency
4. **Feature engineering**: StringIndexer + VectorAssembler — same concept as LabelEncoder + column selection
5. **Three models**: LR (baseline) → RF (non-linear) → GBT (best accuracy)
6. **Evaluation**: AUC, F1, confusion matrix — same metrics, computed in parallel
7. **CrossValidator**: GridSearchCV equivalent — parallelized across cluster

The Big Picture

HDFS → MapReduce → Hive → Spark → **MLlib**

From *storing* data to *predicting* outcomes — the complete big data pipeline.

Instructor Notes

Let us bring it all together. These seven takeaways capture everything we covered today. The first one is the theme of the entire lecture: same math, different execution. If you remember nothing else, remember that. The second through seventh points cover the practical skills: the Pipeline API for organizing your workflow, StringIndexer and VectorAssembler for feature engineering, three classifiers with a clear progression strategy, evaluation metrics you already know, and CrossValidator for distributed hyperparameter tuning. Now look at the big picture at the bottom. We started the semester with HDFS — just storing data. Then MapReduce for processing. Hive for querying. Spark for analyzing. And now MLlib for predicting. You have gone from raw storage to machine learning predictions. That is the complete big data pipeline, and you can build every piece of it. That is a powerful skill set.

What's Next

Wednesday — Hands-On Lab (Session 9B)

- Build the complete arrest prediction pipeline yourself
- Data prep → Feature engineering → RF → LR → GBT → Compare
- Experiment with CrossValidator and hyperparameter tuning

Week 10: Apache Kafka + Spark Structured Streaming

- Shift from **batch** to **streaming** processing
- Kafka architecture: Brokers, Topics, Producers, Consumers
- Spark Structured Streaming: real-time analytics on infinite data

The Journey So Far

Store (HDFS) → Process (MapReduce) → Query (Hive) → Analyze (Spark) → **Predict (MLlib)** → *Stream (Kafka, Week 10)*

Instructor Notes

Here is what is coming up next. On Wednesday, you will get your hands dirty in the lab building the entire arrest prediction pipeline from scratch. You will go through every step we covered today: data preparation, feature engineering, training three models, comparing them, and experimenting with CrossValidator. I strongly recommend reviewing today's slides before the lab so you can focus on coding rather than re-learning concepts. Then in Week 10, we make the final leap from batch processing to streaming. Up until now, everything we have done processes data that already exists. Kafka and Structured Streaming process data as it arrives in real time — imagine predicting arrests as crime reports come in. Look at the journey at the bottom: store, process, query, analyze, predict, and soon stream. You are almost at the end of the big data pipeline. Great work today, and I will see you Wednesday for the lab.